

Access

Smart contract access utilities and implementations

 Copy Markdown Open in Claude

This directory provides ways to restrict who can access the functions of a contract or when they can do it.

- `AccessManager` is a full-fledged access control solution for smart contract systems. Allows creating and assigning multiple hierarchical roles with execution delays for each account across various contracts.
- `AccessManaged` delegates its access control to an authority that dictates the permissions of the managed contract. It's compatible with an `AccessManager` as an authority.
- `AccessControl` provides a per-contract role based access control mechanism. Multiple hierarchical roles can be created and assigned each to multiple accounts within the same instance.
- `Ownable` is a simpler mechanism with a single owner "role" that can be assigned to a single account. This simpler mechanism can be useful for quick tests but projects with production concerns are likely to outgrow it.

Core

`Ownable`

[IAccessControl](#)[AccessControl](#)

Extensions

[IAccessControlEnumerable](#)[AccessControlEnumerable](#)[IAccessControlDefaultAdminRules](#)[AccessControlDefaultAdminRules](#)

AccessManager

[IAuthority](#)[IAccessManager](#)[AccessManager](#)[IAccessManaged](#)[AccessManaged](#)[AuthorityUtils](#)[AccessControl](#)



Contract module that allows children to implement role-based access control mechanisms.

This is a lightweight version that doesn't allow enumerating role members except through off-chain means by accessing the contract event logs. Some applications may benefit from on-chain enumerability, for those cases see [AccessControlEnumerable](#).

Roles are referred to by their `bytes32` identifier. These should be exposed in the external API and be unique. The best way to achieve this is by using `public constant` hash digests:

```
bytes32 public constant MY_ROLE = keccak256("MY_ROLE");
```



Roles can be used to represent a set of permissions. To restrict access to a function call, use [AccessControl.hasRole](#):

```
function foo() public {  
    require(hasRole(MY_ROLE, msg.sender));  
    ...  
}
```



Roles can be granted and revoked dynamically via the [AccessControl.grantRole](#) and [AccessControl.revokeRole](#) functions. Each role has an associated admin role, and only accounts that have a role's admin role can call [AccessControl.grantRole](#) and [AccessControl.revokeRole](#).

By default, the admin role for all roles is `DEFAULT_ADMIN_ROLE`, which means that only accounts with this role will be able to grant or revoke other roles. More complex role relationships can be created by using [AccessControl._setRoleAdmin](#).

 The `DEFAULT_ADMIN_ROLE` is also its own admin: it has permission to grant and revoke this role. Extra precautions should be taken to secure accounts that have been granted it. We recommend using [AccessControlDefaultAdminRules](#) to enforce additional security measures for this role.

Modifiers

Functions

- [supportsInterface\(interfaceId\)](#)
- [hasRole\(role, account\)](#)
- [_checkRole\(role\)](#)
- [_checkRole\(role, account\)](#)
- [getRoleAdmin\(role\)](#)
- [grantRole\(role, account\)](#)
- [revokeRole\(role, account\)](#)
- [renounceRole\(role, callerConfirmation\)](#)
- [_setRoleAdmin\(role, adminRole\)](#)
- [_grantRole\(role, account\)](#)
- [_revokeRole\(role, account\)](#)
- [DEFAULT_ADMIN_ROLE\(\)](#)

Events

- ▶ `IAccessControl`

Errors

- ▶ `IAccessControl`

onlyRole(bytes32 role)

internal #



Modifier that checks that an account has a specific role. Reverts with an `IAccessControl.AccessControlUnauthorizedAccount` error including the required role.

supportsInterface(bytes4 interfaceId) → bool

public #

Returns true if this contract implements the interface defined by `interfaceId`. See the corresponding [ERC section](#) to learn more about how these ids are created.

This function call must use less than 30 000 gas.

hasRole(bytes32 role, address account) → bool

public #

Returns `true` if `account` has been granted `role`.

_checkRole(bytes32 role)

internal #

Reverts with an `IAccessControl.AccessControlUnauthorizedAccount` error if `_msgSender()` is missing `role`. Overriding this function changes the behavior of the `AccessControl.onlyRole` modifier.

_checkRole(bytes32 role, address account)

internal #

Reverts with an `IAccessControl.AccessControlUnauthorizedAccount` error if `account` is missing `role`.

getRoleAdmin(bytes32 role) → bytes32

public #

Returns the admin role that controls `role`. See `AccessControl.grantRole` and `AccessControl.revokeRole`.

To change a role's admin, use `AccessControl._setRoleAdmin`.



`grantRole(bytes32 role, address account)`

public #

Grants `role` to `account`.

If `account` had not been already granted `role`, emits a `IAccessControl.RoleGranted` event.

Requirements:

- the caller must have `role`'s admin role.

May emit a `IAccessControl.RoleGranted` event.

`revokeRole(bytes32 role, address account)`

public #

Revokes `role` from `account`.

If `account` had been granted `role`, emits a `IAccessControl.RoleRevoked` event.

Requirements:

- the caller must have `role`'s admin role.

May emit a `IAccessControl.RoleRevoked` event.

`renounceRole(bytes32 role, address callerConfirmation)`

public #

Revokes `role` from the calling account.

Roles are often managed via `AccessControl.grantRole` and `AccessControl.revokeRole`: this function's purpose is to provide a mechanism for accounts to lose their privileges if they



If the calling account had been revoked `role`, emits a `IAccessControl.RoleRevoked` event.

Requirements:

- the caller must be `callerConfirmation`.

May emit a `IAccessControl.RoleRevoked` event.

`_setRoleAdmin(bytes32 role, bytes32 adminRole)`

internal #

Sets `adminRole` as `role`'s admin role.

Emits a `IAccessControl.RoleAdminChanged` event.

`_grantRole(bytes32 role, address account) → bool`

internal #

Attempts to grant `role` to `account` and returns a boolean indicating if `role` was granted.

Internal function without access restriction.

May emit a `IAccessControl.RoleGranted` event.

`_revokeRole(bytes32 role, address account) → bool`

internal #

Attempts to revoke `role` from `account` and returns a boolean indicating if `role` was revoked.

Internal function without access restriction.

May emit a `IAccessControl.RoleRevoked` event.

IAccessControl



```
import "@openzeppelin/contracts/access/IAccessControl.sol";
```



External interface of AccessControl declared to support ERC-165 detection.

Functions

- [hasRole\(role, account\)](#)
- [getRoleAdmin\(role\)](#)
- [grantRole\(role, account\)](#)
- [revokeRole\(role, account\)](#)
- [renounceRole\(role, callerConfirmation\)](#)

Events

- [RoleAdminChanged\(role, previousAdminRole, newAdminRole\)](#)
- [RoleGranted\(role, account, sender\)](#)
- [RoleRevoked\(role, account, sender\)](#)

Errors

hasRole(bytes32 role, address account) → bool

external #

Returns `true` if `account` has been granted `role`.

getRoleAdmin(bytes32 role) → bytes32

external #

Returns the admin role that controls `role`. See [AccessControl.grantRole](#) and [AccessControl.revokeRole](#).

To change a role's admin, use [AccessControl._setRoleAdmin](#).

grantRole(bytes32 role, address account)

external #

Grants `role` to `account`.

If `account` had not been already granted `role`, emits a [IAccessControl.RoleGranted](#) event.

Requirements:

- the caller must have `role`'s admin role.

revokeRole(bytes32 role, address account)

external #

Revokes `role` from `account`.

If `account` had been granted `role`, emits a [IAccessControl.RoleRevoked](#) event.

Requirements:

OpenZeppelin Docs

- the caller must have `role`'s admin role.

renounceRole(bytes32 role, address callerConfirmation)

external #

Revokes `role` from the calling account.

Roles are often managed via [AccessControl.grantRole](#) and [AccessControl.revokeRole](#): this function's purpose is to provide a mechanism for accounts to lose their privileges if they are compromised (such as when a trusted device is misplaced).

If the calling account had been granted `role`, emits a [IAccessControl.RoleRevoked](#) event.

Requirements:

- the caller must be `callerConfirmation`.

RoleAdminChanged(bytes32 indexed role, bytes32 indexed previousAdminRole, bytes32 indexed newAdminRole)

event #

Emitted when `newAdminRole` is set as `role`'s admin role, replacing `previousAdminRole`

`DEFAULT_ADMIN_ROLE` is the starting admin for all roles, despite

[IAccessControl.RoleAdminChanged](#) not being emitted to signal this.

RoleGranted(bytes32 indexed role, address indexed account, address indexed sender)

event #

Emitted when `account` is granted `role`.

`sender` is the account that originated the contract call. This account bears the admin role (for the granted role). Expected in cases where the role was granted using the internal

RoleRevoked(bytes32 indexed role, address indexed account, address indexed sender) event #

Emitted when `account` is revoked `role`.

`sender` is the account that originated the contract call:

- if using `revokeRole`, it is the admin role bearer
- if using `renounceRole`, it is the role bearer (i.e. `account`)

AccessControlUnauthorizedAccount(address account, bytes32 neededRole) error #

The `account` is missing a role.

AccessControlBadConfirmation() error #

The caller of a function is not the expected one.

 Don't confuse with [IAccessControl.AccessControlUnauthorizedAccount](#).

Ownable



```
import "@openzeppelin/contracts/access/Ownable.sol";
```



 **OpenZeppelin Docs** Contract module which provides a basic access control mechanism, where there is an account (an owner) that can be granted exclusive access to specific functions.

The initial owner is set to the address provided by the deployer. This can later be changed with `Ownable.transferOwnership`.

This module is used through inheritance. It will make available the modifier `onlyOwner`, which can be applied to your functions to restrict their use to the owner.

Modifiers

- [onlyOwner\(\)](#)

Functions

- [constructor\(initialOwner\)](#)
- [owner\(\)](#)
- [_checkOwner\(\)](#)
- [renounceOwnership\(\)](#)
- [transferOwnership\(newOwner\)](#)
- [_transferOwnership\(newOwner\)](#)

Events

- [OwnershipTransferred\(previousOwner, newOwner\)](#)

Errors

- [OwnableUnauthorizedAccount\(account\)](#)

onlyOwner()

internal #

Throws if called by any account other than the owner.

constructor(address initialOwner)

internal #

Initializes the contract setting the address provided by the deployer as the initial owner.

owner() → address

public #

Returns the address of the current owner.

_checkOwner()

internal #

Throws if the sender is not the owner.

renounceOwnership()

public #

Leaves the contract without owner. It will not be possible to call `onlyOwner` functions. Can only be called by the current owner.

i Renouncing ownership will leave the contract without an owner, thereby disabling any functionality that is only available to the owner.

transferOwnership(address newOwner)

public #

 **OpenZeppelin Docs** Transfers ownership of the contract to a new account (`newOwner`). Can only be called by the current owner.

`_transferOwnership(address newOwner)`

internal #

Transfers ownership of the contract to a new account (`newOwner`). Internal function without access restriction.

`OwnershipTransferred(address indexed previousOwner, address indexed newOwner)`

event #

`OwnableUnauthorizedAccount(address account)`

error #

The caller account is not authorized to perform an operation.

`OwnableInvalidOwner(address owner)`

error #

The owner is not a valid owner account. (eg. `address(0)`)

Ownable2Step



```
import "@openzeppelin/contracts/access/Ownable2Step.sol";
```



Contract module which provides access control mechanism, where there is an account (an owner) that can be granted exclusive access to specific functions.

This extension of the `Ownable` contract includes a two-step mechanism to transfer ownership, where the new owner must call `Ownable2Step.acceptOwnership` in order to replace the old one.

This can help prevent common mistakes, such as transfers of ownership to incorrect accounts, or to contracts that are unable to interact with the permission system.

The initial owner is specified at deployment time in the constructor for `Ownable`. This can later be changed with `Ownable.transferOwnership` and `Ownable2Step.acceptOwnership`.

This module is used through inheritance. It will make available all functions from parent (`Ownable`).

Functions

- [`pendingOwner\(\)`](#)
- [`transferOwnership\(newOwner\)`](#)
- [`\_transferOwnership\(newOwner\)`](#)
- [`acceptOwnership\(\)`](#)

► `Ownable`

Events

- [`OwnershipTransferStarted\(previousOwner, newOwner\)`](#)

► `Ownable`

Errors

► `Ownable`

`pendingOwner()` → `address`

public #

 Returns the address of the pending owner.

transferOwnership(address newOwner)

public #

Starts the ownership transfer of the contract to a new account. Replaces the pending transfer if there is one. Can only be called by the current owner.

Setting `newOwner` to the zero address is allowed; this can be used to cancel an initiated ownership transfer.

_transferOwnership(address newOwner)

internal #

Transfers ownership of the contract to a new account (`newOwner`) and deletes any pending owner. Internal function without access restriction.

acceptOwnership()

public #

The new owner accepts the ownership transfer.

OwnershipTransferStarted(address indexed previousOwner, address indexed newOwner)

event #

AccessControlDefaultAdminRules



```
import "@openzeppelin/contracts/access/extensions/AccessControlDefaultAdminRules.sol" 
```

 Extension of `AccessControl` that allows specifying special rules to manage the `DEFAULT_ADMIN_ROLE` holder, which is a sensitive role with special permissions over other roles that may potentially have privileged rights in the system.

If a specific role doesn't have an admin role assigned, the holder of the `DEFAULT_ADMIN_ROLE` will have the ability to grant it and revoke it.

This contract implements the following risk mitigations on top of `AccessControl`:

- Only one account holds the `DEFAULT_ADMIN_ROLE` since deployment until it's potentially renounced.
- Enforces a 2-step process to transfer the `DEFAULT_ADMIN_ROLE` to another account.
- Enforces a configurable delay between the two steps, with the ability to cancel before the transfer is accepted.
- The delay can be changed by scheduling, see `AccessControlDefaultAdminRules.changeDefaultAdminDelay`.
- Role transfers must wait at least one block after scheduling before it can be accepted.
- It is not possible to use another role to manage the `DEFAULT_ADMIN_ROLE`.

Example usage:

```
contract MyToken is AccessControlDefaultAdminRules {  
  constructor() AccessControlDefaultAdminRules(  
    3 days,  
    msg.sender // Explicit initial `DEFAULT_ADMIN_ROLE` holder  
  ) {}  
}
```



Functions

- [`constructor\(initialDelay, initialDefaultAdmin\)`](#)
- [`supportsInterface\(interfaceId\)`](#)
- [`owner\(\)`](#)
- [`grantRole\(role, account\)`](#)

- 
- [revokeRole\(role, account\)](#)
 - [renounceRole\(role, account\)](#)
 - [_grantRole\(role, account\)](#)

- [_revokeRole\(role, account\)](#)
- [_setRoleAdmin\(role, adminRole\)](#)
- [defaultAdmin\(\)](#)
- [pendingDefaultAdmin\(\)](#)
- [defaultAdminDelay\(\)](#)
- [pendingDefaultAdminDelay\(\)](#)
- [defaultAdminDelayIncreaseWait\(\)](#)
- [beginDefaultAdminTransfer\(newAdmin\)](#)
- [_beginDefaultAdminTransfer\(newAdmin\)](#)
- [cancelDefaultAdminTransfer\(\)](#)
- [_cancelDefaultAdminTransfer\(\)](#)
- [acceptDefaultAdminTransfer\(\)](#)
- [_acceptDefaultAdminTransfer\(\)](#)
- [changeDefaultAdminDelay\(newDelay\)](#)
- [_changeDefaultAdminDelay\(newDelay\)](#)
- [rollbackDefaultAdminDelay\(\)](#)
- [_rollbackDefaultAdminDelay\(\)](#)
- [_delayChangeWait\(newDelay\)](#)

► AccessControl

Events

- IAccessControlDefaultAdminRules
- IAccessControl

► [IAccessControlDefaultAdminRules](#)

► [IAccessControl](#)

constructor(uint48 initialDelay, address initialDefaultAdmin) internal #

Sets the initial values for [AccessControlDefaultAdminRules.defaultAdminDelay](#) and [AccessControlDefaultAdminRules.defaultAdmin](#) address.

supportsInterface(bytes4 interfaceId) → bool public #

owner() → address public #

Gets the address of the owner.

grantRole(bytes32 role, address account) public #

See [AccessControl.grantRole](#). Reverts for `DEFAULT_ADMIN_ROLE`.

revokeRole(bytes32 role, address account) public #

See [AccessControl.revokeRole](#). Reverts for `DEFAULT_ADMIN_ROLE`.

renounceRole(bytes32 role, address account) public #

See [AccessControl.renounceRole](#).



For the `DEFAULT_ADMIN_ROLE`, it only allows renouncing in two steps by first calling

`AccessControlDefaultAdminRules.beginDefaultAdminTransfer` to the `address(0)`, so it's

required that the `AccessControlDefaultAdminRules.pendingDefaultAdmin` schedule has also passed when calling this function.

After its execution, it will not be possible to call `onlyRole(DEFAULT_ADMIN_ROLE)` functions.

- Renouncing `DEFAULT_ADMIN_ROLE` will leave the contract without a `AccessControlDefaultAdminRules.defaultAdmin`, thereby disabling any functionality that is only available for it, and the possibility of reassigning a non-administrated role.

`_grantRole(bytes32 role, address account) → bool`

internal #

See [AccessControl._grantRole](#).

For `DEFAULT_ADMIN_ROLE`, it only allows granting if there isn't already a `AccessControlDefaultAdminRules.defaultAdmin` or if the role has been previously renounced.

- Exposing this function through another mechanism may make the `DEFAULT_ADMIN_ROLE` assignable again. Make sure to guarantee this is the expected behavior in your implementation.

`_revokeRole(bytes32 role, address account) → bool`

internal #

Attempts to revoke `role` from `account` and returns a boolean indicating if `role` was revoked.

Internal function without access restriction.

May emit a [IAccessControl.RoleRevoked](#) event.

See [AccessControl._setRoleAdmin](#). Reverts for `DEFAULT_ADMIN_ROLE`.

defaultAdmin() → `address`

public #

Returns the address of the current `DEFAULT_ADMIN_ROLE` holder.

pendingDefaultAdmin() → `address newAdmin, uint48 schedule`

public #

Returns a tuple of a `newAdmin` and an accept schedule.

After the `schedule` passes, the `newAdmin` will be able to accept the `AccessControlDefaultAdminRules.defaultAdmin` role by calling `AccessControlDefaultAdminRules.acceptDefaultAdminTransfer`, completing the role transfer.

A zero value only in `acceptSchedule` indicates no pending admin transfer.

i A zero address `newAdmin` means that `AccessControlDefaultAdminRules.defaultAdmin` is being renounced.

defaultAdminDelay() → `uint48`

public #

Returns the delay required to schedule the acceptance of a `AccessControlDefaultAdminRules.defaultAdmin` transfer started.

This delay will be added to the current timestamp when calling `AccessControlDefaultAdminRules.beginDefaultAdminTransfer` to set the acceptance schedule.

If `defaultAdminDelay` has been scheduled, it will take effect as soon as the schedule passes, making this function returns the new delay. See

[AccessControlDefaultAdminRules.changeDefaultAdminDelay](#).

pendingDefaultAdminDelay() → uint48 newDelay, uint48 schedule

public #

Returns a tuple of `newDelay` and an effect schedule.

After the `schedule` passes, the `newDelay` will get into effect immediately for every new

[AccessControlDefaultAdminRules.defaultAdmin](#) transfer started with

[AccessControlDefaultAdminRules.beginDefaultAdminTransfer](#).

A zero value only in `effectSchedule` indicates no pending delay change.

i A zero value only for `newDelay` means that the next [AccessControlDefaultAdminRules.defaultAdminDelay](#) will be zero after the effect schedule.

defaultAdminDelayIncreaseWait() → uint48

public #

Maximum time in seconds for an increase to

[AccessControlDefaultAdminRules.defaultAdminDelay](#) (that is scheduled using

[AccessControlDefaultAdminRules.changeDefaultAdminDelay](#)) to take effect. Default to 5

days.

When the [AccessControlDefaultAdminRules.defaultAdminDelay](#) is scheduled to be

increased, it goes into effect after the new delay has passed with the purpose of giving enough time for reverting any accidental change (i.e. using milliseconds instead of seconds) that may lock the contract. However, to avoid excessive schedules, the wait is capped by this function and it can be overridden for a custom

[AccessControlDefaultAdminRules.defaultAdminDelay](#) increase scheduling.

Make sure to wait a reasonable amount of time while overriding this value, otherwise, there's a risk of setting a high new delay that goes into effect almost immediately without the possibility of human intervention in the case of an input error (eg. set milliseconds instead of seconds).

beginDefaultAdminTransfer(address newAdmin)

public #

Starts a `AccessControlDefaultAdminRules.defaultAdmin` transfer by setting a `AccessControlDefaultAdminRules.pendingDefaultAdmin` scheduled for acceptance after the current timestamp plus a `AccessControlDefaultAdminRules.defaultAdminDelay`.

Requirements:

- Only can be called by the current `AccessControlDefaultAdminRules.defaultAdmin`.

Emits a `IAccessControlDefaultAdminRules.DefaultAdminTransfersScheduled` event.

_beginDefaultAdminTransfer(address newAdmin)

internal #

See `AccessControlDefaultAdminRules.beginDefaultAdminTransfer`.

Internal function without access restriction.

cancelDefaultAdminTransfer()

public #

Cancels a `AccessControlDefaultAdminRules.defaultAdmin` transfer previously started with `AccessControlDefaultAdminRules.beginDefaultAdminTransfer`.

A `AccessControlDefaultAdminRules.pendingDefaultAdmin` not yet accepted can also be cancelled with this function.

Requirements:

 Only can be called by the current `AccessControlDefaultAdminRules.defaultAdmin`.

May emit a `IAccessControlDefaultAdminRules.DefaultAdminTransferCanceled` event.

`_cancelDefaultAdminTransfer()`

internal #

See `AccessControlDefaultAdminRules.cancelDefaultAdminTransfer`.

Internal function without access restriction.

`acceptDefaultAdminTransfer()`

public #

Completes a `AccessControlDefaultAdminRules.defaultAdmin` transfer previously started with `AccessControlDefaultAdminRules.beginDefaultAdminTransfer`.

After calling the function:

- `DEFAULT_ADMIN_ROLE` should be granted to the caller.
- `DEFAULT_ADMIN_ROLE` should be revoked from the previous holder.
- `AccessControlDefaultAdminRules.pendingDefaultAdmin` should be reset to zero values.

Requirements:

- Only can be called by the `AccessControlDefaultAdminRules.pendingDefaultAdmin`'s `newAdmin`.
- The `AccessControlDefaultAdminRules.pendingDefaultAdmin`'s `acceptSchedule` should've passed.

`_acceptDefaultAdminTransfer()`

internal #

See `AccessControlDefaultAdminRules.acceptDefaultAdminTransfer`.

`changeDefaultAdminDelay(uint48 newDelay)`
`public #`

Initiates a `AccessControlDefaultAdminRules.defaultAdminDelay` update by setting a `AccessControlDefaultAdminRules.pendingDefaultAdminDelay` scheduled for getting into effect after the current timestamp plus a `AccessControlDefaultAdminRules.defaultAdminDelay`.

This function guarantees that any call to `AccessControlDefaultAdminRules.beginDefaultAdminTransfer` done between the timestamp this method is called and the `AccessControlDefaultAdminRules.pendingDefaultAdminDelay` effect schedule will use the current `AccessControlDefaultAdminRules.defaultAdminDelay` set before calling.

The `AccessControlDefaultAdminRules.pendingDefaultAdminDelay`'s effect schedule is defined in a way that waiting until the schedule and then calling `AccessControlDefaultAdminRules.beginDefaultAdminTransfer` with the new delay will take at least the same as another `AccessControlDefaultAdminRules.defaultAdmin` complete transfer (including acceptance).

The schedule is designed for two scenarios:

- When the delay is changed for a larger one the schedule is `block.timestamp + newDelay` capped by `AccessControlDefaultAdminRules.defaultAdminDelayIncreaseWait`.
- When the delay is changed for a shorter one, the schedule is `block.timestamp + (current delay - new delay)`.

A `AccessControlDefaultAdminRules.pendingDefaultAdminDelay` that never got into effect will be canceled in favor of a new scheduled change.

Requirements:

- Only can be called by the current `AccessControlDefaultAdminRules.defaultAdmin`.

 **OpenZeppelin Docs**

Emits a `IAccessControlDefaultAdminRules.DefaultAdminDelayChangeScheduled` event and may emit a `IAccessControlDefaultAdminRules.DefaultAdminDelayChangeCanceled` event.

`_changeDefaultAdminDelay(uint48 newDelay)`

internal #

See [AccessControlDefaultAdminRules.changeDefaultAdminDelay](#).

Internal function without access restriction.

`rollbackDefaultAdminDelay()`

public #

Cancels a scheduled [AccessControlDefaultAdminRules.defaultAdminDelay](#) change.

Requirements:

- Only can be called by the current [AccessControlDefaultAdminRules.defaultAdmin](#).

May emit a [IAccessControlDefaultAdminRules.DefaultAdminDelayChangeCanceled](#) event.

`_rollbackDefaultAdminDelay()`

internal #

See [AccessControlDefaultAdminRules.rollbackDefaultAdminDelay](#).

Internal function without access restriction.

`_delayChangeWait(uint48 newDelay) → uint48`

internal #

Returns the amount of seconds to wait after the `newDelay` will become the new [AccessControlDefaultAdminRules.defaultAdminDelay](#).



The value returned guarantees that if the delay is reduced, it will go into effect after a wait that honors the previously set delay.

See [AccessControlDefaultAdminRules.defaultAdminDelayIncreaseWait](#).

AccessControlEnumerable



```
import "@openzeppelin/contracts/access/extensions/AccessControlEnumerable.sol";
```



Extension of [AccessControl](#) that allows enumerating the members of each role.

Functions

- [supportsInterface\(interfaceId\)](#)
- [getRoleMember\(role, index\)](#)
- [getRoleMemberCount\(role\)](#)
- [getRoleMembers\(role\)](#)
- [_grantRole\(role, account\)](#)
- [_revokeRole\(role, account\)](#)

► [AccessControl](#)

Events

► [IAccessControl](#)

Errors

```
supportsInterface(bytes4 interfaceId) → bool
```

public #

```
getRoleMember(bytes32 role, uint256 index) → address
```

public #

Returns one of the accounts that have `role`. `index` must be a value between 0 and `AccessControlEnumerable.getRoleMemberCount`, non-inclusive.

Role bearers are not sorted in any particular way, and their ordering may change at any point.

 When using `AccessControlEnumerable.getRoleMember` and `AccessControlEnumerable.getRoleMemberCount`, make sure you perform all queries on the same block. See the following [forum post](#) for more information.

```
getRoleMemberCount(bytes32 role) → uint256
```

public #

Returns the number of accounts that have `role`. Can be used together with `AccessControlEnumerable.getRoleMember` to enumerate all bearers of a role.

```
getRoleMembers(bytes32 role) → address[]
```

public #

Return all accounts that have `role`

 This operation will copy the entire storage to memory, which can be quite expensive. This is designed to mostly be used by view accessors that are queried without any gas fees. Developers should keep in mind that this function has an unbounded cost, and using it as part of a state-changing function may render the function uncallable if the set grows to a point where copying to memory consumes too much gas to fit in a block.

`grantRole(bytes32 role, address account) → bool` internal #

 **OpenZeppelin Docs**

Overload `AccessControl._grantRole` to track enumerable memberships

`_revokeRole(bytes32 role, address account) → bool` internal #

Overload `AccessControl._revokeRole` to track enumerable memberships

IAccessControlDefaultAdminRules



```
import "@openzeppelin/contracts/access/extensions/IAccessControlDefaultAdminRules.sol"
```

External interface of AccessControlDefaultAdminRules declared to support ERC-165 detection.

Functions

- [defaultAdmin\(\)](#)
- [pendingDefaultAdmin\(\)](#)
- [defaultAdminDelay\(\)](#)
- [pendingDefaultAdminDelay\(\)](#)
- [beginDefaultAdminTransfer\(newAdmin\)](#)
- [cancelDefaultAdminTransfer\(\)](#)
- [acceptDefaultAdminTransfer\(\)](#)
- [changeDefaultAdminDelay\(newDelay\)](#)
- [rollbackDefaultAdminDelay\(\)](#)



▶ `IAccessControl`

Events

- [`DefaultAdminTransferScheduled\(newAdmin, acceptSchedule\)`](#)
- [`DefaultAdminTransferCanceled\(\)`](#)
- [`DefaultAdminDelayChangeScheduled\(newDelay, effectSchedule\)`](#)
- [`DefaultAdminDelayChangeCanceled\(\)`](#)

▶ `IAccessControl`

Errors

- [`AccessControlInvalidDefaultAdmin\(defaultAdmin\)`](#)
- [`AccessControlEnforcedDefaultAdminRules\(\)`](#)
- [`AccessControlEnforcedDefaultAdminDelay\(schedule\)`](#)

▶ `IAccessControl`

`defaultAdmin()` → `address`

external #

Returns the address of the current `DEFAULT_ADMIN_ROLE` holder.

`pendingDefaultAdmin()` → `address newAdmin, uint48 acceptSchedule`

external #

Returns a tuple of a `newAdmin` and an accept schedule.

After the `schedule` passes, the `newAdmin` will be able to accept the `AccessControlDefaultAdminRules.defaultAdmin` role by calling `AccessControlDefaultAdminRules.acceptDefaultAdminTransfer`, completing the role transfer.

A zero value only in `acceptSchedule` indicates no pending admin transfer.

i A zero address `newAdmin` means that `AccessControlDefaultAdminRules.defaultAdmin` is being renounced.

defaultAdminDelay() → uint48

external #

Returns the delay required to schedule the acceptance of a `AccessControlDefaultAdminRules.defaultAdmin` transfer started.

This delay will be added to the current timestamp when calling `AccessControlDefaultAdminRules.beginDefaultAdminTransfer` to set the acceptance schedule.

i If a delay change has been scheduled, it will take effect as soon as the schedule passes, making this function returns the new delay. See `AccessControlDefaultAdminRules.changeDefaultAdminDelay`.

pendingDefaultAdminDelay() → uint48 newDelay, uint48 effectSchedule

external #

Returns a tuple of `newDelay` and an effect schedule.

After the `schedule` passes, the `newDelay` will get into effect immediately for every new `AccessControlDefaultAdminRules.defaultAdmin` transfer started with `AccessControlDefaultAdminRules.beginDefaultAdminTransfer`.

A zero value only in `effectSchedule` indicates no pending delay change.



`newDelay` means that the next `AccessControlDefaultAdminRules.defaultAdminDelay` will be zero after the effect schedule.

`beginDefaultAdminTransfer(address newAdmin)`

external #

Starts a `AccessControlDefaultAdminRules.defaultAdmin` transfer by setting a `AccessControlDefaultAdminRules.pendingDefaultAdmin` scheduled for acceptance after the current timestamp plus a `AccessControlDefaultAdminRules.defaultAdminDelay`.

Requirements:

- Only can be called by the current `AccessControlDefaultAdminRules.defaultAdmin`.

Emits a `IAccessControlDefaultAdminRules.DefaultAdminTransferScheduled` event.

`cancelDefaultAdminTransfer()`

external #

Cancels a `AccessControlDefaultAdminRules.defaultAdmin` transfer previously started with `AccessControlDefaultAdminRules.beginDefaultAdminTransfer`.

A `AccessControlDefaultAdminRules.pendingDefaultAdmin` not yet accepted can also be cancelled with this function.

Requirements:

- Only can be called by the current `AccessControlDefaultAdminRules.defaultAdmin`.

May emit a `IAccessControlDefaultAdminRules.DefaultAdminTransferCanceled` event.

`acceptDefaultAdminTransfer()`

external #

 **OpenZeppelin Docs** Completes a `AccessControlDefaultAdminRules.defaultAdmin` transfer previously started with `AccessControlDefaultAdminRules.beginDefaultAdminTransfer`.

After calling the function:

- `DEFAULT_ADMIN_ROLE` should be granted to the caller.
- `DEFAULT_ADMIN_ROLE` should be revoked from the previous holder.
- `AccessControlDefaultAdminRules.pendingDefaultAdmin` should be reset to zero values.

Requirements:

- Only can be called by the `AccessControlDefaultAdminRules.pendingDefaultAdmin`'s `newAdmin`.
- The `AccessControlDefaultAdminRules.pendingDefaultAdmin`'s `acceptSchedule` should've passed.

changeDefaultAdminDelay(uint48 newDelay)

external #

Initiates a `AccessControlDefaultAdminRules.defaultAdminDelay` update by setting a `AccessControlDefaultAdminRules.pendingDefaultAdminDelay` scheduled for getting into effect after the current timestamp plus a `AccessControlDefaultAdminRules.defaultAdminDelay`.

This function guarantees that any call to `AccessControlDefaultAdminRules.beginDefaultAdminTransfer` done between the timestamp this method is called and the `AccessControlDefaultAdminRules.pendingDefaultAdminDelay` effect schedule will use the current `AccessControlDefaultAdminRules.defaultAdminDelay` set before calling.

The `AccessControlDefaultAdminRules.pendingDefaultAdminDelay`'s effect schedule is defined in a way that waiting until the schedule and then calling `AccessControlDefaultAdminRules.beginDefaultAdminTransfer` with the new delay will take at

least the same as another `AccessControlDefaultAdminRules.defaultAdmin` complete transfer (including acceptance).

The schedule is designed for two scenarios:

- When the delay is changed for a larger one the schedule is `block.timestamp + newDelay` capped by `AccessControlDefaultAdminRules.defaultAdminDelayIncreaseWait`.
- When the delay is changed for a shorter one, the schedule is `block.timestamp + (current delay - new delay)`.

A `AccessControlDefaultAdminRules.pendingDefaultAdminDelay` that never got into effect will be canceled in favor of a new scheduled change.

Requirements:

- Only can be called by the current `AccessControlDefaultAdminRules.defaultAdmin`.

Emits a `IAccessControlDefaultAdminRules.DefaultAdminDelayChangeScheduled` event and may emit a `IAccessControlDefaultAdminRules.DefaultAdminDelayChangeCanceled` event.

`rollbackDefaultAdminDelay()`

external #

Cancels a scheduled `AccessControlDefaultAdminRules.defaultAdminDelay` change.

Requirements:

- Only can be called by the current `AccessControlDefaultAdminRules.defaultAdmin`.

May emit a `IAccessControlDefaultAdminRules.DefaultAdminDelayChangeCanceled` event.

`defaultAdminDelayIncreaseWait() → uint48`

external #



Maximum time in seconds for an increase to

OpenZeppelin Docs`AccessControlDefaultAdminRules.defaultAdminDelay` (that is scheduled using`AccessControlDefaultAdminRules.changeDefaultAdminDelay`) to take effect. Default to 5

days.

When the `AccessControlDefaultAdminRules.defaultAdminDelay` is scheduled to be increased, it goes into effect after the new delay has passed with the purpose of giving enough time for reverting any accidental change (i.e. using milliseconds instead of seconds) that may lock the contract. However, to avoid excessive schedules, the wait is capped by this function and it can be overridden for a custom

`AccessControlDefaultAdminRules.defaultAdminDelay` increase scheduling.

Make sure to add a reasonable amount of time while overriding this value, otherwise, there's a risk of setting a high new delay that goes into effect almost immediately without the possibility of human intervention in the case of an input error (eg. set milliseconds instead of seconds).

DefaultAdminTransferScheduled(address indexed newAdmin, uint48 acceptSchedule)

event #

Emitted when a `AccessControlDefaultAdminRules.defaultAdmin` transfer is started, setting `newAdmin` as the next address to become the `AccessControlDefaultAdminRules.defaultAdmin` by calling `AccessControlDefaultAdminRules.acceptDefaultAdminTransfer` only after `acceptSchedule` passes.

DefaultAdminTransferCanceled()

event #

Emitted when a `AccessControlDefaultAdminRules.pendingDefaultAdmin` is reset if it was never accepted, regardless of its schedule.

OpenZeppelin Docs `AccessControlDefaultAdminRules.setDefaultAdminDelay(uint48 newDelay, uint48 effectSchedule)` event

Emitted when a `AccessControlDefaultAdminRules.defaultAdminDelay` change is started, setting `newDelay` as the next delay to be applied between default admin transfer after `effectSchedule` has passed.

`DefaultAdminDelayChangeCanceled()` event

Emitted when a `AccessControlDefaultAdminRules.pendingDefaultAdminDelay` is reset if its schedule didn't pass.

`AccessControlInvalidDefaultAdmin(address defaultAdmin)` error

The new default admin is not a valid default admin.

`AccessControlEnforcedDefaultAdminRules()` error

At least one of the following rules was violated:

- The `DEFAULT_ADMIN_ROLE` must only be managed by itself.
- The `DEFAULT_ADMIN_ROLE` must only be held by one account at the time.
- Any `DEFAULT_ADMIN_ROLE` transfer must be in two delayed steps.

`AccessControlEnforcedDefaultAdminDelay(uint48 schedule)` error

The delay for transferring the default admin delay is enforced and the operation must wait until `schedule`.

IAccessControlEnumerable



```
import "@openzeppelin/contracts/access/extensions/IAccessControlEnumerable.sol";
```



External interface of AccessControlEnumerable declared to support ERC-165 detection.

Functions

- [getRoleMember\(role, index\)](#)
- [getRoleMemberCount\(role\)](#)

► IAccessControl

Events

► IAccessControl

Errors

► IAccessControl

getRoleMember(bytes32 role, uint256 index) → address

external #

Returns one of the accounts that have `role`. `index` must be a value between 0 and `AccessControlEnumerable.getRoleMemberCount`, non-inclusive.



⚠ When using `AccessControlEnumerable.getRoleMember` and `AccessControlEnumerable.getRoleMemberCount`, make sure you perform all queries on the same block. See the following [forum post](#) for more information.

getRoleMemberCount(bytes32 role) → uint256

external #

Returns the number of accounts that have `role`. Can be used together with `AccessControlEnumerable.getRoleMember` to enumerate all bearers of a role.

AccessManaged



```
import "@openzeppelin/contracts/access/manager/AccessManaged.sol";
```



This contract module makes available a `AccessManaged.restricted` modifier. Functions decorated with this modifier will be permissioned according to an "authority": a contract like `AccessManager` that follows the `IAuthority` interface, implementing a policy that allows certain callers to access certain functions.

⚠ The `restricted` modifier should never be used on `internal` functions, judiciously used in `public` functions, and ideally only used in `external` functions. See `AccessManaged.restricted`.

Modifiers

Functions

- [`constructor\(initialAuthority\)`](#)
- [`authority\(\)`](#)
- [`setAuthority\(newAuthority\)`](#)
- [`isConsumingScheduledOp\(\)`](#)
- [`\_setAuthority\(newAuthority\)`](#)
- [`\_checkCanCall\(caller, data\)`](#)

Events

- ▶ `IAccessManaged`

Errors

- ▶ `IAccessManaged`

`restricted()`

internal #

Restricts access to a function as defined by the connected Authority for this contract and the caller and selector of the function that entered the contract.



The `external` modifier should only be used on `external` functions. It is okay to use it on `public` functions that are used as external entry points and are not called internally. Unless you know what you're doing, it should never be used on `internal` functions. Failure to follow these rules can have critical security implications! This is because the permissions are determined by the function that entered the contract, i.e. the function at the bottom of the call stack, and not the function where the modifier is visible in the source code.

⚠️ Avoid adding this modifier to the `receive()` function or the `fallback()`. These functions are the only execution paths where a function selector cannot be unambiguously determined from the calldata since the selector defaults to `0x00000000` in the `receive()` function and similarly in the `fallback()` function if no calldata is provided. (See [AccessManaged._checkCanCall](#)).

The `receive()` function will always panic whereas the `fallback()` may panic depending on the calldata length.

constructor(address initialAuthority)

internal #

Initializes the contract connected to an initial authority.

authority() → address

public #

Returns the current authority.

setAuthority(address newAuthority)

public #

Transfers control to a new authority. The caller must be the current authority.

isConsumingScheduledOp() → bytes4

public #



Returns true only in the context of a delayed restricted call, at the moment that the scheduled operation is being consumed. Prevents denial of service for delayed restricted calls in the case that the contract performs attacker controlled calls.

`_setAuthority(address newAuthority)`

internal #

Transfers control to a new authority. Internal function with no access restriction. Allows bypassing the permissions set by the current authority.

`_checkCanCall(address caller, bytes data)`

internal #

Reverts if the caller is not allowed to call the function identified by a selector. Panics if the calldata is less than 4 bytes long.

AccessManager



```
import "@openzeppelin/contracts/access/manager/AccessManager.sol";
```



AccessManager is a central contract to store the permissions of a system.

A smart contract under the control of an AccessManager instance is known as a target, and will inherit from the `AccessManaged` contract, be connected to this contract as its manager and implement the `AccessManaged.restricted` modifier on a set of functions selected to be permissioned. Note that any function without this setup won't be effectively restricted.

The restriction rules for such functions are defined in terms of "roles" identified by an `uint64` and scoped by target (`address`) and function selectors (`bytes4`). These roles are stored in

this contract and can be configured by admins (`ADMIN_ROLE` members) after a delay (see `AccessManager.getTargetAdminDelay`).

For each target contract, admins can configure the following without any delay:

- The target's `AccessManaged.authority` via `AccessManager.updateAuthority`.
- Close or open a target via `AccessManager.setTargetClosed` keeping the permissions intact.
- The roles that are allowed (or disallowed) to call a given function (identified by its selector) through `AccessManager.setTargetFunctionRole`.

By default every address is member of the `PUBLIC_ROLE` and every target function is restricted to the `ADMIN_ROLE` until configured otherwise. Additionally, each role has the following configuration options restricted to this manager's admins:

- A role's admin role via `AccessManager.setRoleAdmin` who can grant or revoke roles.
- A role's guardian role via `AccessManager.setRoleGuardian` who's allowed to cancel operations.
- A delay in which a role takes effect after being granted through `AccessManager.setGrantDelay`.
- A delay of any target's admin action via `AccessManager.setTargetAdminDelay`.
- A role label for discoverability purposes with `AccessManager.labelRole`.

Any account can be added and removed into any number of these roles by using the `AccessControl.grantRole` and `AccessControl.revokeRole` functions restricted to each role's admin (see `AccessControl.getRoleAdmin`).

Since all the permissions of the managed system can be modified by the admins of this instance, it is expected that they will be highly secured (e.g., a multisig or a well-configured DAO).

OpenZeppelin Docs This contract implements a form of the `IAuthority` interface, but `AccessManager.canCall` has additional return data so it doesn't inherit `IAuthority`. It is however compatible with the `IAuthority` interface since the first 32 bytes of the return data are a boolean as expected by that interface.

i Systems that implement other access control mechanisms (for example using `Ownable`) can be paired with an `AccessManager` by transferring permissions (ownership in the case of `Ownable`) directly to the `AccessManager`. Users will be able to interact with these contracts through the `AccessManager.execute` function, following the access rules registered in the `AccessManager`. Keep in mind that in that context, the `msg.sender` seen by restricted functions will be `AccessManager` itself.

⚠ When granting permissions over an `Ownable` or `AccessControl` contract to an `AccessManager`, be very mindful of the danger associated with functions such as `Ownable.renounceOwnership` or `AccessControl.renounceRole`.

Modifiers

- `onlyAuthorized()`

Functions

- `constructor(initialAdmin)`
- `canCall(caller, target, selector)`
- `expiration()`
- `minSetback()`
- `isTargetClosed(target)`
- `getTargetFunctionRole(target, selector)`
- `getTargetAdminDelay(target)`
- `getRoleAdmin(roleId)`
- `getRoleGuardian(roleId)`
- `getRoleGrantDelay(roleId)`

 OpenZeppelin Docs

- [getAccess\(roleId, account\)](#)
- [hasRole\(roleId, account\)](#)
- [labelRole\(roleId, label\)](#)
- [grantRole\(roleId, account, executionDelay\)](#)
- [revokeRole\(roleId, account\)](#)
- [renounceRole\(roleId, callerConfirmation\)](#)
- [setRoleAdmin\(roleId, admin\)](#)
- [setRoleGuardian\(roleId, guardian\)](#)
- [setGrantDelay\(roleId, newDelay\)](#)
- [_grantRole\(roleId, account, grantDelay, executionDelay\)](#)
- [_revokeRole\(roleId, account\)](#)
- [_setRoleAdmin\(roleId, admin\)](#)
- [_setRoleGuardian\(roleId, guardian\)](#)
- [_setGrantDelay\(roleId, newDelay\)](#)
- [setTargetFunctionRole\(target, selectors, roleId\)](#)
- [_setTargetFunctionRole\(target, selector, roleId\)](#)
- [setTargetAdminDelay\(target, newDelay\)](#)
- [_setTargetAdminDelay\(target, newDelay\)](#)
- [setTargetClosed\(target, closed\)](#)
- [_setTargetClosed\(target, closed\)](#)
- [getSchedule\(id\)](#)
- [getNonce\(id\)](#)
- [schedule\(target, data, when\)](#)
- [execute\(target, data\)](#)
- [cancel\(caller, target, data\)](#)
- [consumeScheduledOp\(caller, data\)](#)
- [_consumeScheduledOp\(operationId\)](#)

- [hashOperation\(caller, target, data\)](#)
- [updateAuthority\(target, newAuthority\)](#)
- [ADMIN_ROLE\(\)](#)

- [PUBLIC_ROLE\(\)](#)

► Multicall

Events

► IAccessManager

Errors

► IAccessManager

onlyAuthorized()

internal #

Check that the caller is authorized to perform the operation. See [AccessManager](#) description for a detailed breakdown of the authorization logic.

constructor(address initialAdmin)

public #

canCall(address caller, address target, bytes4 selector) → bool
immediate, uint32 delay

public #

Check if an address ([caller](#)) is authorised to call a given function on a given contract directly (with no restriction). Additionally, it returns the delay needed to perform the call indirectly through the [AccessManager.schedule](#) & [AccessManager.execute](#) workflow.



This function is usually called by the targeted contract to control immediate execution of restricted functions. Therefore we only return true if the call can be performed without any delay. If the call is subject to a previously set delay (not zero), then the function should return false and the caller should schedule the operation for future execution.

If `allowed` is true, the delay can be disregarded and the operation can be immediately executed, otherwise the operation can be executed if and only if delay is greater than 0.

i The `IAuthority` interface does not include the `uint32` delay. This is an extension of that interface that is backward compatible. Some contracts may thus ignore the second return argument. In that case they will fail to identify the indirect workflow, and will consider calls that require a delay to be forbidden.

i This function does not report the permissions of the admin functions in the manager itself. These are defined by the [AccessManager](#) documentation.

expiration() → uint32

public #

Expiration delay for scheduled proposals. Defaults to 1 week.

⚠ Avoid overriding the expiration with 0. Otherwise every contract proposal will be expired immediately, disabling any scheduling usage.

minSetback() → uint32

public #

Minimum setback for all delay updates, with the exception of execution delays. It can be increased without setback (and reset via [AccessControl.revokeRole](#) in the event of an accidental increase). Defaults to 5 days.

isTargetClosed(address target) → bool

public #

 **OpenZeppelin Docs** Get whether the contract is closed disabling any access. Otherwise role permissions are applied.

 When the manager itself is closed, admin functions are still accessible to avoid locking the contract.

getTargetFunctionRole(address target, bytes4 selector) → uint64

public #

Get the role required to call a function.

getTargetAdminDelay(address target) → uint32

public #

Get the admin delay for a target contract. Changes to contract configuration are subject to this delay.

getRoleAdmin(uint64 roleId) → uint64

public #

Get the id of the role that acts as an admin for the given role.

The admin permission is required to grant the role, revoke the role and update the execution delay to execute an operation that is restricted to this role.

getRoleGuardian(uint64 roleId) → uint64

public #

Get the role that acts as a guardian for a given role.

The guardian permission allows canceling operations that have been scheduled under the role.

getRoleGrantDelay(uint64 roleId) → uint32

public #



Get the role current grant delay.
 Its value may change at any point without an event emitted following a call to `AccessManager.setGrantDelay`. Changes to this value, including effect timepoint are notified in advance by the `IAccessManager.RoleGrantDelayChanged` event.

`getAccess(uint64 roleId, address account) → uint48 since, uint32 currentDelay, uint32 pendingDelay, uint48 effect`

public #

Get the access details for a given account for a given role. These details include the timepoint at which membership becomes active, and the delay applied to all operations by this user that requires this permission level.

Returns: [0] Timestamp at which the account membership becomes valid. 0 means role is not granted. [1] Current execution delay for the account. [2] Pending execution delay for the account. [3] Timestamp at which the pending execution delay will become active. 0 means no delay update is scheduled.

`hasRole(uint64 roleId, address account) → bool isMember, uint32 executionDelay`

public #

Check if a given account currently has the permission level corresponding to a given role. Note that this permission might be associated with an execution delay.

`AccessManager.getAccess` can provide more details.

`labelRole(uint64 roleId, string label)`

public #

Give a label to a role, for improved role discoverability by UIs.

Requirements:

- the caller must be a global admin



`roleId` must not be the `ADMIN_ROLE` or `PUBLIC_ROLE`

Emits a `IAccessManager.RoleLabel` event.

`grantRole(uint64 roleId, address account, uint32 executionDelay)`

public #

Add `account` to `roleId`, or change its execution delay.

This gives the account the authorization to call any function that is restricted to this role. An optional execution delay (in seconds) can be set. If that delay is non 0, the user is required to schedule any operation that is restricted to members of this role. The user will only be able to execute the operation after the delay has passed, before it has expired. During this period, admin and guardians can cancel the operation (see `AccessManager.cancel`).

If the account has already been granted this role, the execution delay will be updated. This update is not immediate and follows the delay rules. For example, if a user currently has a delay of 3 hours, and this is called to reduce that delay to 1 hour, the new delay will take some time to take effect, enforcing that any operation executed in the 3 hours that follows this update was indeed scheduled before this update.

Requirements:

- the caller must be an admin for the role (see `AccessControl.getRoleAdmin`)
- granted role must not be the `PUBLIC_ROLE`

Emits a `IAccessControl.RoleGranted` event.

`revokeRole(uint64 roleId, address account)`

public #

Remove an account from a role, with immediate effect. If the account does not have the role, this call has no effect.

Requirements: OpenZeppelin Docs

- the caller must be an admin for the role (see [AccessControl.getRoleAdmin](#))

- revoked role must not be the `PUBLIC_ROLE`

Emits a [IAccessControl.RoleRevoked](#) event if the account had the role.

renounceRole(uint64 roleId, address callerConfirmation)

public #

Renounce role permissions for the calling account with immediate effect. If the sender is not in the role this call has no effect.

Requirements:

- the caller must be `callerConfirmation`.

Emits a [IAccessControl.RoleRevoked](#) event if the account had the role.

setRoleAdmin(uint64 roleId, uint64 admin)

public #

Change admin role for a given role.

Requirements:

- the caller must be a global admin
- `roleId` must not be the `ADMIN_ROLE` or `PUBLIC_ROLE`

Emits a [IAccessControl.RoleAdminChanged](#) event

setRoleGuardian(uint64 roleId, uint64 guardian)

public #

Change guardian role for a given role.

Requirements:

OpenZeppelin Docs

- the caller must be a global admin
- `roleId` must not be the `ADMIN_ROLE` or `PUBLIC_ROLE`

Emits a `IAccessManager.RoleGuardianChanged` event

setGrantDelay(uint64 roleId, uint32 newDelay)

public #

Update the delay for granting a `roleId`.

Requirements:

- the caller must be a global admin
- `roleId` must not be the `PUBLIC_ROLE`

Emits a `IAccessManager.RoleGrantDelayChanged` event.

_grantRole(uint64 roleId, address account, uint32 grantDelay, uint32 executionDelay) → bool

internal #

Internal version of `AccessControl.grantRole` without access control. Returns true if the role was newly granted.

Emits a `IAccessControl.RoleGranted` event.

_revokeRole(uint64 roleId, address account) → bool

internal #

Internal version of `AccessControl.revokeRole` without access control. This logic is also used by `AccessControl.renounceRole`. Returns true if the role was previously granted.

 Emits a `IAccessControl.RoleRevoked` event if the account had the role.

`_setRoleAdmin(uint64 roleId, uint64 admin)`

internal #

Internal version of `AccessManager.setRoleAdmin` without access control.

Emits a `IAccessControl.RoleAdminChanged` event.

 Setting the admin role as the `PUBLIC_ROLE` is allowed, but it will effectively allow anyone to set grant or revoke such role.

`_setRoleGuardian(uint64 roleId, uint64 guardian)`

internal #

Internal version of `AccessManager.setRoleGuardian` without access control.

Emits a `IAccessManager.RoleGuardianChanged` event.

 Setting the guardian role as the `PUBLIC_ROLE` is allowed, but it will effectively allow anyone to cancel any scheduled operation for such role.

`_setGrantDelay(uint64 roleId, uint32 newDelay)`

internal #

Internal version of `AccessManager.setGrantDelay` without access control.

Emits a `IAccessManager.RoleGrantDelayChanged` event.

`setTargetFunctionRole(address target, bytes4[] selectors, uint64 roleId)` public #

Set the role required to call functions identified by the `selectors` in the `target` contract.

Requirements:



Emits a [IAccessManager.TargetFunctionRoleUpdated](#) event per selector.

`_setTargetFunctionRole(address target, bytes4 selector, uint64 roleId)` internal #

Internal version of [AccessManager.setTargetFunctionRole](#) without access control.

Emits a [IAccessManager.TargetFunctionRoleUpdated](#) event.

`setTargetAdminDelay(address target, uint32 newDelay)` public #

Set the delay for changing the configuration of a given target contract.

Requirements:

- the caller must be a global admin

Emits a [IAccessManager.TargetAdminDelayUpdated](#) event.

`_setTargetAdminDelay(address target, uint32 newDelay)` internal #

Internal version of [AccessManager.setTargetAdminDelay](#) without access control.

Emits a [IAccessManager.TargetAdminDelayUpdated](#) event.

`setTargetClosed(address target, bool closed)` public #

Set the closed flag for a contract.

Closing the manager itself won't disable access to admin methods to avoid locking the contract.

Requirements: OpenZeppelin Docs

- the caller must be a global admin

Emits a `IAccessManager.TargetClosed` event.

`_setTargetClosed(address target, bool closed)`

internal #

Set the closed flag for a contract. This is an internal setter with no access restrictions.

Emits a `IAccessManager.TargetClosed` event.

`getSchedule(bytes32 id) → uint48`

public #

Return the timepoint at which a scheduled operation will be ready for execution. This returns 0 if the operation is not yet scheduled, has expired, was executed, or was canceled.

`getNonce(bytes32 id) → uint32`

public #

Return the nonce for the latest scheduled operation with a given id. Returns 0 if the operation has never been scheduled.

`schedule(address target, bytes data, uint48 when) → bytes32 operationId, uint32 nonce`

public #

Schedule a delayed operation for future execution, and return the operation identifier. It is possible to choose the timestamp at which the operation becomes executable as long as it satisfies the execution delays required for the caller. The special value zero will automatically set the earliest possible time.



Returns the `operationId` that was scheduled. Since this value is a hash of the parameters, it can reoccur when the same parameters are used; if this is relevant, the returned `nonce` can be used to uniquely identify this scheduled operation from other occurrences of the same `operationId` in invocations of `AccessManager.execute` and `AccessManager.cancel`.

Emits a `IAccessManager.OperationScheduled` event.

i It is not possible to concurrently schedule more than one operation with the same `target` and `data`. If this is necessary, a random byte can be appended to `data` to act as a salt that will be ignored by the target contract if it is using standard Solidity ABI encoding.

execute(address target, bytes data) → uint32

public #

Execute a function that is delay restricted, provided it was properly scheduled beforehand, or the execution delay is 0.

Returns the nonce that identifies the previously scheduled operation that is executed, or 0 if the operation wasn't previously scheduled (if the caller doesn't have an execution delay).

Emits an `IAccessManager.OperationExecuted` event only if the call was scheduled and delayed.

cancel(address caller, address target, bytes data) → uint32

public #

Cancel a scheduled (delayed) operation. Returns the nonce that identifies the previously scheduled operation that is cancelled.

Requirements:

- the caller must be the proposer, a guardian of the targeted function, or a global admin

 Emits a `IAccessManager.OperationCanceled` event.

`consumeScheduledOp(address caller, bytes data)`

public #

Consume a scheduled operation targeting the caller. If such an operation exists, mark it as consumed (emit an `IAccessManager.OperationExecuted` event and clean the state). Otherwise, throw an error.

This is useful for contracts that want to enforce that calls targeting them were scheduled on the manager, with all the verifications that it implies.

Emit a `IAccessManager.OperationExecuted` event.

`_consumeScheduledOp(bytes32 operationId) → uint32`

internal #

Internal variant of `AccessManager.consumeScheduledOp` that operates on bytes32 operationId.

Returns the nonce of the scheduled operation that is consumed.

`hashOperation(address caller, address target, bytes data) → bytes32`

public #

Hashing function for delayed operations.

`updateAuthority(address target, address newAuthority)`

public #

Changes the authority of a target managed by this manager instance.

Requirements:

- the caller must be a global admin

The identifier of the admin role. Required to perform most configuration operations including other roles' management and target restrictions.

PUBLIC_ROLE() → uint64

public #

The identifier of the public role. Automatically granted to all addresses with no delay.

AuthorityUtils



```
import "@openzeppelin/contracts/access/manager/AuthorityUtils.sol";
```



Functions

- [canCallWithDelay\(authority, caller, target, selector\)](#)

canCallWithDelay(address authority, address caller, address target, bytes4 selector) → bool immediate, uint32 delay

internal #

Since `AccessManager` implements an extended `IAuthority` interface, invoking `canCall` with backwards compatibility for the preexisting `IAuthority` interface requires special care to avoid reverting on insufficient return data. This helper function takes care of invoking `canCall` in a backwards compatible way without reverting.



```
import "@openzeppelin/contracts/access/manager/IAccessManaged.sol";
```



Functions

- [authority\(\)](#)
- [setAuthority\(\)](#)
- [isConsumingScheduledOp\(\)](#)

Events

- [AuthorityUpdated\(authority\)](#)

Errors

- [AccessManagedUnauthorized\(caller\)](#)
- [AccessManagedRequiredDelay\(caller, delay\)](#)
- [AccessManagedInvalidAuthority\(authority\)](#)

authority() → address

external #

Returns the current authority.

Transfers control to a new authority. The caller must be the current authority.

isConsumingScheduledOp() → bytes4

external #

Returns true only in the context of a delayed restricted call, at the moment that the scheduled operation is being consumed. Prevents denial of service for delayed restricted calls in the case that the contract performs attacker controlled calls.

AuthorityUpdated(address authority)

event #

Authority that manages this contract was updated.

AccessManagedUnauthorized(address caller)

error #

AccessManagedRequiredDelay(address caller, uint32 delay)

error #

AccessManagedInvalidAuthority(address authority)

error #

IAccessManager



```
import "@openzeppelin/contracts/access/manager/IAccessManager.sol";
```



Functions

OpenZeppelin Docs

- [canCall\(caller, target, selector\)](#)
- [expiration\(\)](#)
- [minSetback\(\)](#)
- [isTargetClosed\(target\)](#)
- [getTargetFunctionRole\(target, selector\)](#)
- [getTargetAdminDelay\(target\)](#)
- [getRoleAdmin\(roleId\)](#)
- [getRoleGuardian\(roleId\)](#)
- [getRoleGrantDelay\(roleId\)](#)
- [getAccess\(roleId, account\)](#)
- [hasRole\(roleId, account\)](#)
- [labelRole\(roleId, label\)](#)
- [grantRole\(roleId, account, executionDelay\)](#)
- [revokeRole\(roleId, account\)](#)
- [renounceRole\(roleId, callerConfirmation\)](#)
- [setRoleAdmin\(roleId, admin\)](#)
- [setRoleGuardian\(roleId, guardian\)](#)
- [setGrantDelay\(roleId, newDelay\)](#)
- [setTargetFunctionRole\(target, selectors, roleId\)](#)
- [setTargetAdminDelay\(target, newDelay\)](#)
- [setTargetClosed\(target, closed\)](#)
- [getSchedule\(id\)](#)
- [getNonce\(id\)](#)
- [schedule\(target, data, when\)](#)
- [execute\(target, data\)](#)
- [cancel\(caller, target, data\)](#)
- [consumeScheduledOp\(caller, data\)](#)

 [hashOperation\(caller, target, data\)](#)
[updateAuthority\(target, newAuthority\)](#)

Events

- [OperationScheduled\(operationId, nonce, schedule, caller, target, data\)](#)
- [OperationExecuted\(operationId, nonce\)](#)
- [OperationCanceled\(operationId, nonce\)](#)
- [RoleLabel\(roleId, label\)](#)
- [RoleGranted\(roleId, account, delay, since, newMember\)](#)
- [RoleRevoked\(roleId, account\)](#)
- [RoleAdminChanged\(roleId, admin\)](#)
- [RoleGuardianChanged\(roleId, guardian\)](#)
- [RoleGrantDelayChanged\(roleId, delay, since\)](#)
- [TargetClosed\(target, closed\)](#)
- [TargetFunctionRoleUpdated\(target, selector, roleId\)](#)
- [TargetAdminDelayUpdated\(target, delay, since\)](#)

Errors

- [AccessManagerAlreadyScheduled\(operationId\)](#)
- [AccessManagerNotScheduled\(operationId\)](#)
- [AccessManagerNotReady\(operationId\)](#)
- [AccessManagerExpired\(operationId\)](#)
- [AccessManagerLockedRole\(roleId\)](#)
- [AccessManagerBadConfirmation\(\)](#)



- [AccessManagerUnauthorizedAccount\(msgsender, roleId\)](#)
- [AccessManagerUnauthorizedCall\(caller, target, selector\)](#)
- [AccessManagerUnauthorizedConsume\(target\)](#)
- [AccessManagerUnauthorizedCancel\(msgsender, caller, target, selector\)](#)
- [AccessManagerInvalidInitialAdmin\(initialAdmin\)](#)

canCall(address caller, address target, bytes4 selector) → bool allowed, ^{external} uint32 delay #

Check if an address (`caller`) is authorised to call a given function on a given contract directly (with no restriction). Additionally, it returns the delay needed to perform the call indirectly through the [AccessManager.schedule](#) & [AccessManager.execute](#) workflow.

This function is usually called by the targeted contract to control immediate execution of restricted functions. Therefore we only return true if the call can be performed without any delay. If the call is subject to a previously set delay (not zero), then the function should return false and the caller should schedule the operation for future execution.

If `allowed` is true, the delay can be disregarded and the operation can be immediately executed, otherwise the operation can be executed if and only if delay is greater than 0.

i The IAuthority interface does not include the `uint32` delay. This is an extension of that interface that is backward compatible. Some contracts may thus ignore the second return argument. In that case they will fail to identify the indirect workflow, and will consider calls that require a delay to be forbidden.

i This function does not report the permissions of the admin functions in the manager itself. These are defined by the [AccessManager](#) documentation.

expiration() → uint32

^{external} #

Expiration delay for scheduled proposals. Defaults to 1 week.

**minSetback()** → **uint32**

external #

Minimum setback for all delay updates, with the exception of execution delays. It can be increased without setback (and reset via `AccessControl.revokeRole` in the event of an accidental increase). Defaults to 5 days.

isTargetClosed(address target) → **bool**

external #

Get whether the contract is closed disabling any access. Otherwise role permissions are applied.

i When the manager itself is closed, admin functions are still accessible to avoid locking the contract.

getTargetFunctionRole(address target, bytes4 selector) → **uint64**

external #

Get the role required to call a function.

getTargetAdminDelay(address target) → **uint32**

external #

Get the admin delay for a target contract. Changes to contract configuration are subject to this delay.

getRoleAdmin(uint64 roleId) → **uint64**

external #

Get the id of the role that acts as an admin for the given role.



The admin permission is required to grant the role, revoke the role and update the execution delay to execute an operation that is restricted to this role.

getRoleGuardian(uint64 roleId) → uint64

external #

Get the role that acts as a guardian for a given role.

The guardian permission allows canceling operations that have been scheduled under the role.

getRoleGrantDelay(uint64 roleId) → uint32

external #

Get the role current grant delay.

Its value may change at any point without an event emitted following a call to `AccessManager.setGrantDelay`. Changes to this value, including effect timepoint are notified in advance by the `IAccessManager.RoleGrantDelayChanged` event.

getAccess(uint64 roleId, address account) → uint48 since, uint32 currentDelay, uint32 pendingDelay, uint48 effect

external #

Get the access details for a given account for a given role. These details include the timepoint at which membership becomes active, and the delay applied to all operations by this user that requires this permission level.

Returns: [0] Timestamp at which the account membership becomes valid. 0 means role is not granted. [1] Current execution delay for the account. [2] Pending execution delay for the account. [3] Timestamp at which the pending execution delay will become active. 0 means no delay update is scheduled.

OpenZeppelin Docs `address account) → bool isMember, uint32 executionDelay` external

Check if a given account currently has the permission level corresponding to a given role. Note that this permission might be associated with an execution delay.

[AccessManager.getAccess](#) can provide more details.

`labelRole(uint64 roleId, string label)` external

Give a label to a role, for improved role discoverability by UIs.

Requirements:

- the caller must be a global admin
- `roleId` must not be the `ADMIN_ROLE` or `PUBLIC_ROLE`

Emits a [IAccessManager.RoleLabel](#) event.

`grantRole(uint64 roleId, address account, uint32 executionDelay)` external

Add `account` to `roleId`, or change its execution delay.

This gives the account the authorization to call any function that is restricted to this role. An optional execution delay (in seconds) can be set. If that delay is non 0, the user is required to schedule any operation that is restricted to members of this role. The user will only be able to execute the operation after the delay has passed, before it has expired. During this period, admin and guardians can cancel the operation (see [AccessManager.cancel](#)).

If the account has already been granted this role, the execution delay will be updated. This update is not immediate and follows the delay rules. For example, if a user currently has a delay of 3 hours, and this is called to reduce that delay to 1 hour, the new delay will



take some time to take effect, enforcing that any operation executed in the 3 hours that follows this update was indeed scheduled before this update.

Requirements:

- the caller must be an admin for the role (see [AccessControl.getRoleAdmin\(\)](#))
- granted role must not be the `PUBLIC_ROLE`

Emits a [IAccessControl.RoleGranted](#) event.

revokeRole(uint64 roleId, address account)

external #

Remove an account from a role, with immediate effect. If the account does not have the role, this call has no effect.

Requirements:

- the caller must be an admin for the role (see [AccessControl.getRoleAdmin\(\)](#))
- revoked role must not be the `PUBLIC_ROLE`

Emits a [IAccessControl.RoleRevoked](#) event if the account had the role.

renounceRole(uint64 roleId, address callerConfirmation)

external #

Renounce role permissions for the calling account with immediate effect. If the sender is not in the role this call has no effect.

Requirements:

- the caller must be `callerConfirmation`.

Emits a [IAccessControl.RoleRevoked](#) event if the account had the role.

OpenZeppelin Docs

external #

Change admin role for a given role.

Requirements:

- the caller must be a global admin
- `roleId` must not be the `ADMIN_ROLE` or `PUBLIC_ROLE`

Emits a [`IAccessControl.RoleAdminChanged`](#) event

setRoleGuardian(uint64 roleId, uint64 guardian)

external #

Change guardian role for a given role.

Requirements:

- the caller must be a global admin
- `roleId` must not be the `ADMIN_ROLE` or `PUBLIC_ROLE`

Emits a [`IAccessManager.RoleGuardianChanged`](#) event

setGrantDelay(uint64 roleId, uint32 newDelay)

external #

Update the delay for granting a `roleId`.

Requirements:

- the caller must be a global admin
- `roleId` must not be the `PUBLIC_ROLE`

Emits a [`IAccessManager.RoleGrantDelayChanged`](#) event.

OpenZeppelin Docs

Set the role required to call functions identified by the `selectors` in the `target` contract.

Requirements:

- the caller must be a global admin

Emits a `IAccessManager.TargetFunctionRoleUpdated` event per selector.

setTargetAdminDelay(address target, uint32 newDelay) external

Set the delay for changing the configuration of a given target contract.

Requirements:

- the caller must be a global admin

Emits a `IAccessManager.TargetAdminDelayUpdated` event.

setTargetClosed(address target, bool closed) external

Set the closed flag for a contract.

Closing the manager itself won't disable access to admin methods to avoid locking the contract.

Requirements:

- the caller must be a global admin

Emits a `IAccessManager.TargetClosed` event.

Return the timepoint at which a scheduled operation will be ready for execution. This returns 0 if the operation is not yet scheduled, has expired, was executed, or was canceled.

getNonce(bytes32 id) → uint32

external #

Return the nonce for the latest scheduled operation with a given id. Returns 0 if the operation has never been scheduled.

schedule(address target, bytes data, uint48 when) → bytes32 operationId, uint32 nonce external #

Schedule a delayed operation for future execution, and return the operation identifier. It is possible to choose the timestamp at which the operation becomes executable as long as it satisfies the execution delays required for the caller. The special value zero will automatically set the earliest possible time.

Returns the `operationId` that was scheduled. Since this value is a hash of the parameters, it can reoccur when the same parameters are used; if this is relevant, the returned `nonce` can be used to uniquely identify this scheduled operation from other occurrences of the same `operationId` in invocations of `AccessManager.execute` and `AccessManager.cancel`.

Emits a `IAccessManager.OperationScheduled` event.

i It is not possible to concurrently schedule more than one operation with the same `target` and `data`. If this is necessary, a random byte can be appended to `data` to act as a salt that will be ignored by the target contract if it is using standard Solidity ABI encoding.

OpenZeppelin Docs `cancel(address caller, address target, bytes data) → uint32`

external #

Execute a function that is delay restricted, provided it was properly scheduled

beforehand, or the execution delay is 0.

Returns the nonce that identifies the previously scheduled operation that is executed, or 0 if the operation wasn't previously scheduled (if the caller doesn't have an execution delay).

Emits an `IAccessManager.OperationExecuted` event only if the call was scheduled and delayed.

cancel(address caller, address target, bytes data) → uint32

external #

Cancel a scheduled (delayed) operation. Returns the nonce that identifies the previously scheduled operation that is cancelled.

Requirements:

- the caller must be the proposer, a guardian of the targeted function, or a global admin

Emits a `IAccessManager.OperationCanceled` event.

consumeScheduledOp(address caller, bytes data)

external #

Consume a scheduled operation targeting the caller. If such an operation exists, mark it as consumed (emit an `IAccessManager.OperationExecuted` event and clean the state).

Otherwise, throw an error.

This is useful for contracts that want to enforce that calls targeting them were scheduled on the manager, with all the verifications that it implies.

Emit a `IAccessManager.OperationExecuted` event.

 **OpenZeppelin Docs** `caller, address target, bytes data) → bytes32` external #

Hashing function for delayed operations.

updateAuthority(address target, address newAuthority) external #

Changes the authority of a target managed by this manager instance.

Requirements:

- the caller must be a global admin

OperationScheduled(bytes32 indexed operationId, uint32 indexed nonce, uint48 schedule, address caller, address target, bytes data) event #

A delayed operation was scheduled.

OperationExecuted(bytes32 indexed operationId, uint32 indexed nonce) event #

A scheduled operation was executed.

OperationCanceled(bytes32 indexed operationId, uint32 indexed nonce) event #

A scheduled operation was canceled.

RoleLabel(uint64 indexed roleId, string label) event #

Informational labelling for a roleId.

RoleGranted(uint64 indexed roleId, address indexed account, uint32 delay, event #
 OpenZeppelin Docs Member)

Emitted when `account` is granted `roleId`.

i The meaning of the `since` argument depends on the `newMember` argument. If the role is granted to a new member, the `since` argument indicates when the account becomes a member of the role, otherwise it indicates the timestamp when the execution delay update takes effect for this account and roleId.

RoleRevoked(uint64 indexed roleId, address indexed account) event #

Emitted when `account` membership or `roleId` is revoked. Unlike granting, revoking is instantaneous.

RoleAdminChanged(uint64 indexed roleId, uint64 indexed admin) event #

Role acting as admin over a given `roleId` is updated.

RoleGuardianChanged(uint64 indexed roleId, uint64 indexed guardian) event #

Role acting as guardian over a given `roleId` is updated.

RoleGrantDelayChanged(uint64 indexed roleId, uint32 delay, uint48 since) event #

Grant delay for a given `roleId` will be updated to `delay` when `since` is reached.

TargetClosed(address indexed target, bool closed) event #

 Target mode is updated (true = closed, false = open).
OpenZeppelin Docs

TargetFunctionRoleUpdated(address indexed target, bytes4 selector, uint64 indexed roleId) event #

Role required to invoke `selector` on `target` is updated to `roleId`.

TargetAdminDelayUpdated(address indexed target, uint32 delay, uint48 since) event #

Admin delay for a given `target` will be updated to `delay` when `since` is reached.

AccessManagerAlreadyScheduled(bytes32 operationId) error #

AccessManagerNotScheduled(bytes32 operationId) error #

AccessManagerNotReady(bytes32 operationId) error #

AccessManagerExpired(bytes32 operationId) error #

AccessManagerLockedRole(uint64 roleId) error #

AccessManagerBadConfirmation() error #

AccessManagerUnauthorizedAccount(address msgsender, uint64 roleId) error #

AccessManagerUnauthorizedCall(address caller, address target, bytes4 selector) error #

AccessManagerUnauthorizedCancel(address msgsender, address caller, address target, bytes4 selector) error #

AccessManagerInvalidInitialAdmin(address initialAdmin) error #

IAuthority



```
import "@openzeppelin/contracts/access/manager/IAuthority.sol";
```



Standard interface for permissioning originally defined in Dappsys.

Functions

- [canCall\(caller, target, selector\)](#)

canCall(address caller, address target, bytes4 selector) → bool allowed external #

Returns true if the caller can invoke on a target the function identified by a function selector.

[< Overview](#)

Previous Page

[Account >](#)

Next Page